

YARN: Yet Another Resource Negotiator

VIT University

September 18, 2025

Introduction to YARN

- YARN stands for Yet Another Resource Negotiator.
- It is a resource management layer for the Hadoop ecosystem.
- Introduced in Hadoop 2.0 to overcome scalability limitations.
- Separates resource management and job scheduling/monitoring.
- Enables multiple data processing engines (MapReduce, Spark).
- Acts as an OS for Hadoop clusters.
- Provides APIs for developing distributed applications.
- Improves cluster utilization and scalability.

Architecture Overview

- ResourceManager (RM) manages global resource allocation.
- NodeManager (NM) manages resources on each node.
- ApplicationMaster (AM) manages execution of a single app.
- Containers are units of resource allocation.
- Clients submit jobs to the ResourceManager.
- AM negotiates resources and monitors tasks.
- Communication via heartbeat messages.
- Decouples resource management from job logic.

ResourceManager Components

- Scheduler: Allocates resources based on policies.
- ApplicationManager: Manages job lifecycles.
- AdminService: Handles RM admin tasks.
- ClientService: Manages client communication.
- RMWebService: Exposes REST API.
- Supports pluggable schedulers (FIFO, Fair, Capacity).
- Maintains job queue and prioritization.
- Ensures fault tolerance and scalability.

NodeManager Responsibilities

- Launches and monitors containers on a node.
- Reports node health to ResourceManager.
- Executes container start/stop commands.
- Provides logs to the AM and RM.
- Manages local storage for containers.
- Communicates regularly via heartbeat.
- Handles security and isolation.
- Tracks resource usage per container.

ApplicationMaster Role

- Manages execution of a single application.
- Requests containers from RM.
- Assigns tasks to containers.
- Tracks task progress and handles failures.
- Releases containers upon completion.
- Reports job status to client.
- Can be custom-built for specific frameworks.
- Runs in a container like other tasks.

YARN Container

- Basic unit of resource allocation.
- Includes memory, CPU, and optional resources.
- Launched by NodeManager on request from AM.
- Isolated using cgroups and Linux containers.
- Can run any arbitrary process.
- Container lifecycle managed by NM.
- Assigned unique container ID.
- Secure execution using user privileges.

YARN vs Classic MapReduce

- YARN decouples job and resource management.
- Classic MR used JobTracker and TaskTracker.
- JobTracker bottlenecked with scalability issues.
- YARN allows multiple AMs to run concurrently.
- Classic MR was limited to MapReduce jobs.
- YARN supports various data processing models.
- Fault tolerance improved in YARN.
- YARN enables better resource utilization.

Scheduler Types in YARN

- FIFO Scheduler: Simple first-in, first-out queue.
- Capacity Scheduler: Multi-tenant, capacity-guaranteed.
- Fair Scheduler: Ensures fair sharing of resources.
- Each supports configurable queues.
- Queues can be hierarchical.
- Applications placed in queues based on user/group.
- Policies support priority and preemption.
- Pluggable scheduler framework.

YARN in Hadoop Ecosystem

- Core of modern Hadoop architecture.
- Supports MapReduce v2, Apache Spark, Tez, etc.
- Enables real-time and batch processing.
- Works with Hive, Pig, and other tools.
- Powers workflows via Oozie.
- Integrated with HDFS and HBase.
- Provides central resource governance.
- Basis for multi-framework support.

YARN Timeline and Evolution

- Introduced in Hadoop 2.x.
- Replaced JobTracker/TaskTracker model.
- Continuously improved in 2.x and 3.x.
- YARN Federation for multi-cluster scaling.
- Support for Docker containers added.
- Enhanced with auto-scaling features.
- Improved scheduling and fault recovery.
- Integral to Big Data platforms like Cloudera and Hortonworks.

YARN Job Lifecycle

- Client submits application to RM.
- RM allocates container for AM.
- AM registers with RM.
- AM requests containers for tasks.
- Tasks run in containers on NMs.
- AM monitors task progress.
- Job completion reported to RM.
- AM and containers cleaned up.

Fault Tolerance in YARN

- RM supports high availability (active-standby).
- AM failures can trigger retries.
- Container failures handled by AM.
- NMs report health to RM.
- Unhealthy nodes blacklisted.
- Jobs rescheduled on failure.
- Log aggregation helps in debugging.
- Keeps cluster operational during partial failures.

YARN Configuration Files

- `yarn-site.xml`: Core YARN configuration.
- `capacity-scheduler.xml`: Scheduler settings.
- `mapred-site.xml`: MapReduce-on-YARN settings.
- Defines resource limits and queues.
- Network and security settings configured here.
- Logging and recovery options set in XML.
- Fine-grained control over container sizes.
- Can be managed via Ambari or Cloudera Manager.

Monitoring and Logging

- YARN Web UI for cluster monitoring.
- ResourceManager UI shows job and resource status.
- NodeManager UI shows per-node container info.
- Logs stored in HDFS for analysis.
- Aggregated logs accessible per application.
- Metrics available via JMX and REST API.
- Third-party tools like Grafana supported.
- Critical for debugging and optimization.

Security in YARN

- Kerberos used for authentication.
- Secure token-based authentication between components.
- Access Control Lists (ACLs) for job security.
- Container isolation via OS-level controls.
- Encrypted communication supported.
- Role-based access to UI and APIs.
- Integration with Hadoop security framework.
- Logs access controlled for privacy.

YARN Federation

- Supports multiple sub-clusters in a federation.
- Each sub-cluster has its own RM.
- Global RM Router distributes load.
- Enhances scalability across data centers.
- Applications can span multiple clusters.
- Improves fault isolation and resiliency.
- Transparent to users and developers.
- Works with existing applications.

YARN and Containers

- YARN can launch Docker containers.
- Provides resource isolation and packaging.
- Container-executor handles Docker run logic.
- Useful for microservices and ML workloads.
- Enables portable and reproducible environments.
- Integration with Kubernetes in newer versions.
- Supports GPU and custom resource types.
- Aligns Hadoop with cloud-native design.

YARN Limitations

- Complexity in configuration and tuning.
- Overhead in launching ApplicationMasters.
- Delay in container allocation in busy clusters.
- Resource fragmentation can occur.
- Requires robust monitoring and logging.
- Troubleshooting is sometimes non-trivial.
- Integration with new tools needs testing.
- Improvements ongoing in newer versions.

Future of YARN

- Integration with container orchestrators.
- Better GPU and ML framework support.
- Dynamic resource sharing and auto-scaling.
- Enhanced fault tolerance and HA.
- Improved developer tools and APIs.
- More lightweight AM and container support.
- Continued evolution alongside Hadoop 4.x.
- Interoperability with cloud-native platforms.

Summary

- YARN is a key component in the Hadoop ecosystem.
- Enables efficient resource management and scaling.
- Decouples job logic from resource allocation.
- Supports diverse frameworks beyond MapReduce.
- Scalable, secure, and extensible design.
- Monitored and configured through powerful tools.
- Continues to evolve with modern workloads.
- Essential for big data platforms in enterprises.